

4. ソフトウェア制作

1. 目的

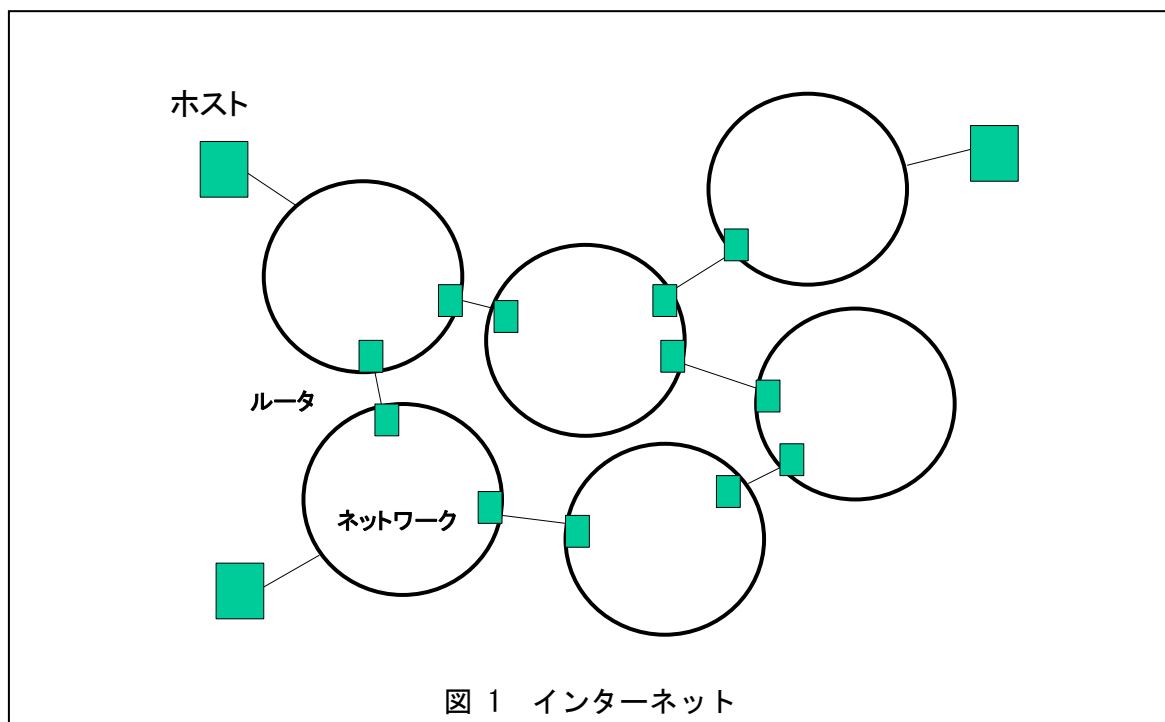
本実験の目的は、実習を通してネットワークプログラミングに関する知識を習得することと、グループワークをおこなう場合の問題点を理解することである。

2. インターネットと Java によるネットワークアプリケーションの作成

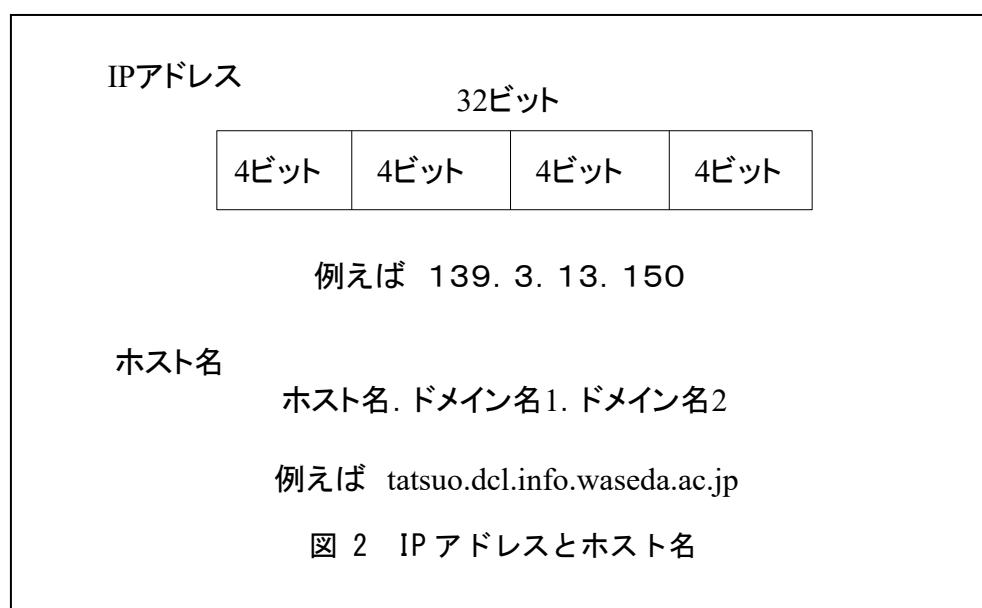
2.1 インターネットとは？

インターネットとは TCP/IP と呼ばれるプロトコルを用いて全世界のネットワークを相互に接続した巨大なコンピュータネットワークである。図 1 に示すように、各ネットワークはルータと呼ばれるコンピュータを介して接続される。その起源は米国防総省の高等研究計画局が始めた分散型コンピュータネットワークの研究プロジェクトである ARPAnet であり、これが 1990 年代中ごろから次第に商用利用されるようになり現在のインターネットになった。

インターネットの初期は、電子メールや NetNews などのアプリケーションにより使用されていたが、WWW と呼ばれるマルチメディアドキュメントシステムの登場により、ビジネスでの利用や家庭からの利用が爆発的に増大し、世界規模の情報通信インフラとしての地位を得るにいたった。インターネット上で利用されるサービスやアプリケーションの大半は TCP/IP と呼ばれる機種に依存しない標準的なプロトコルを利用することにより、アプリケーションをどこのベンダが作ったかに依存せずにお互いに通信することが可能となる。



インターネット上のすべてのコンピュータは、IP アドレスという番号が割り当てられ、この数字を頼りに特定のコンピュータに接続し、情報のやり取りをしている。IP アドレスは 32 ビットで構成される。8 ビットずつピリオドで区切り、それぞれの数値は 0 から 255 の間の整数値を取る。例えば「192.168.0.1」や「216.239.35.100」などである。IP アドレスを 32 ビットとしているのは IPv4 という規格である。現在はインターネットの利用者、用途の増大に伴い IP アドレスの枯渇が問題になり、IP アドレスを 128 ビットに拡張した IPv6 という規格も登場してきている。単純に 32 ビットで指定できる数は約 43 億、128 ビットではこれが実に約 3.8×10^{38} 、地球の全人口を 60 億と仮定すると一人あたり約 5.7×10^{28} になる。



インターネット上において、「IP アドレス」を管理している団体が存在するので、同じ「IP アドレス」を割り当てられているコンピュータが同時に 2 台以上存在することはない。そのおかげで世界中の膨大な数のコンピュータ同士が相手を間違えることなく情報をやり取りすることができるようになる。ただし、IP アドレスは覚えにくいので実際は「ドメイン名」というものが使われる。「waseda.jp」や、「cs.waseda.ac.jp」のように実際にインターネットでホームページにアクセスするときに打ち込むアドレスが実は「ドメイン名」である。ドメイン名だけでは特定のコンピュータを割り出すことはできないので実際はサブドメイン+ドメイン名を入力する。サブドメインとは、早稲田大学の WWW サーバ「www.waseda.ac.jp」でいうと「www」の部分である。サブドメイン+ドメイン名は、IP アドレスと基本的には 1 対 1 で対応している。つまり「www.waseda.ac.jp」は IP アドレス「133.9.4.217」と同じコンピュータを表すということです。ためしに、ブラウザの URL を入力する欄に「http://133.9.4.217」と入力して接続してみると、早稲田大学のホ

ームページが表示される。これは、DNS という仕組みによって実現されている。つまり、IP アドレスさえわかればドメイン名が割り当てられていなくても特定のコンピュータに接続することができる。

インターネットの重要性が増すに従い、インターネットを用いたプログラムの作成技術に関する知識が必須のものとなってきた。つまり、より多くのソフトウェアがインターネット対応となってきたため、ソフトウェア技術者にとってネットワークプログラミングを作成はコンピュータ科学を専攻する学生は全員可能であることが好ましい。

2.2 ネットワークプログラミングとは？

本実験では、Java 言語を用いたネットワークプログラミングの基礎に関して学ぶ。はじめに、自分が使用しているコンピュータの IP アドレスとホスト名を取得するプログラムを以下に示す。

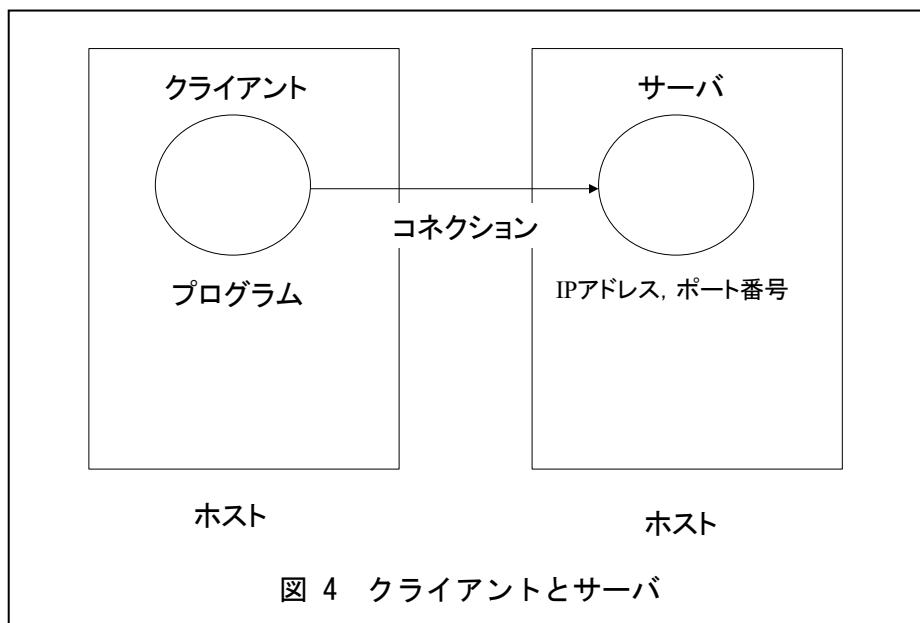
```
import java.net.*;

public class WhoAmI {
    public static void main(String[] args)
        throws Exception {
        if(args.length != 1) {
            System.err.println(
                "Usage: WhoAmI MachineName");
            System.exit(1);
        }
        InetAddress addr
            = InetAddress.getByName(args[0]);
        System.out.println(addr);
    }
}
```

上記のプログラムでは、`InetAddress.getByName` と呼ばれるメソッドを用いてプログラムを実行するコンピュータの IP アドレスとホスト名を取り出すことができる。本プログラムは以下のように実行することができる。

```
> javac WhoAmI.java
> java WhoAmI 自分のホスト名
```

ネットワーク上で通信を行う場合、自分(のコンピュータ)と相手(のコンピュータ)の間で接続を確立する必要がある。このネットワークにおける接続を表す概念が「ソケット」である。ソケットとは、「受け口・軸受け」等の意味であり、イメージの通り自分を相手に接続する（あるいはその逆）受け口となる。



ソケットを利用してプログラム間の通信をおこなう際、プログラムはクライアントとサーバと呼ばれる 2 つのプログラムに分類される。また、2 つのプログラムが通信をおこなうためには、2 つのプログラム間にコネクションを設定する必要がある。2 つのプログラムは、コネクションを介してデータをやり取りすることで、お互いに通信することが可能となる。クライアントとサーバの違いは、サーバはコネクション設定要求を待ち、クライアントはコネクション設定要求を送ることである。

2 つのプログラムが通信する場合、IP アドレスだけではなくポート番号という新しい概念が登場する。ポート番号は IP アドレスで識別されるコンピュータ内のプログラムを区別するために導入された概念である。各コンピュータ内のネットワーク通信をおこなうプログラムは異なるポート番号が割当てられる。ポート番号は 16 ビットの自然数として表現され、メールサーバや Web サーバなどは基本的にすべてのコンピュータであらかじめ決められたポート番号が利用される。

クライアントとサーバが通信する場合、クライアントはサーバとコネクションの接続をおこなう必要がある。コネクションを設定する前に、サーバは以下に示す ServerSocket

のコンストラクタを用いて自分が使用するポート番号を指定する。これは、クライアントが指定されたポート番号に送られたコネクション設定要求を受け取ることを意味する。

```
servSocket = new ServerSocket(waitPort);
```

次に、以下に示すように、ソケットクラスの `accept` メソッドを呼び出すことによりクライアントからのコネクション設定要求を待つ。

```
Socket comSock = null;  
// (略)  
comSock = servSocket.accept();
```

コネクション設定後は、Java の I/O クラスを用いて両方向の通信をおこなうことが可能である。クライアントプログラムは、ソケットの生成とコネクションと設定を `Socket` クラスのコンストラクタを用いておこなう。

```
Socket socket = new Socket(serverIPAddress, serverPortNumber);
```

クライアント、サーバの両方ともソケットクラスの `close` メソッドを呼び出すことによりコネクションを解放することができる。

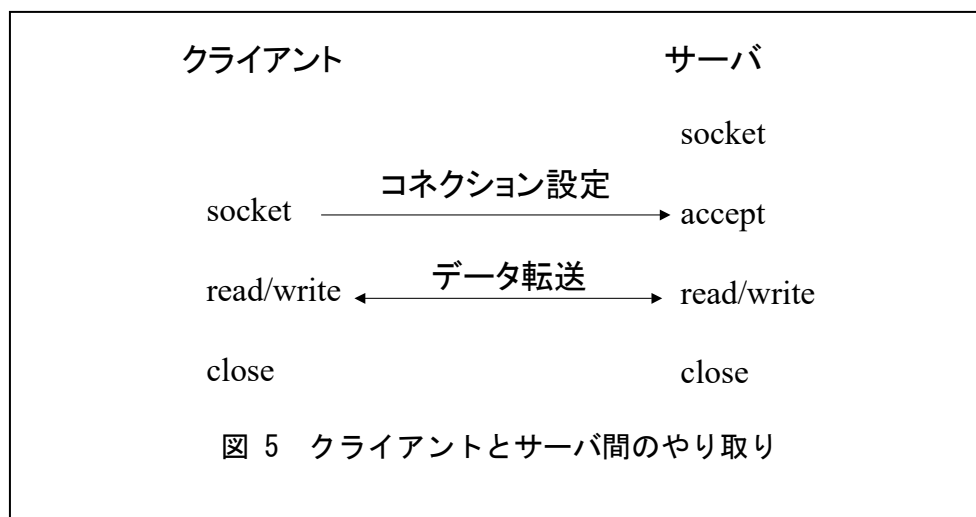


図 4 に、以上解説したクライアントとサーバ間のやり取りをまとめる。

2.3 ネットワークプログラミングの例

本節では、実際に具体的なネットワークプログラミングの例を示す。

JabberServer は、クライアントからの 8080 番ポートに対するコネクション設定要求を受け付け、送られたデータを受け取り、そのデータをそのままクライアントに送り返すプログラムである。本サーバは、クライアントから“END”という文字を受け取ることにより終了する。

JabberClient は、localhost という名前を持つコンピュータ上で動作する、8080 番ポート（図 6 ではサーバプログラムの静的変数である **PORT** を使用）に対するコネクション設定要求を待つサーバプログラムに対してコネクション設定要求を送る。コネクション設定後 10 回文字列を送り、最後に“END”という文字をサーバに転送する。

次に、本節で述べたクライアントとサーバプログラムの実行方法について述べる。はじめに、2 つのコマンドインタプリタを実行するウインドウを開く。1 つ目のウインドウでは以下に示すように、サーバプログラムの実行をおこなう。

```
➤ javac JabberServer.java
➤ java JabberServer
```

2 つ目のウインドウでは、以下に示すようにクライアントプログラムの実行をおこなう。

```
➤ javac JabberClient.java
➤ java JabberClient
```

例として使用するプログラムのソースコードを図 5 と図 6 に示す。

2.4 ネットワークプログラミングの課題

実験の 1 日目は以下の課題に取り組む。

- (1) WhoAmI.java, JabberClient.java, JabberServer.java を実際に打ち込み実行せよ。ホスト名は localhost を使うこと。
- (2) JabberClient, JabberServer において使用するポート番号をプログラムの引数として与えられるようにせよ。

3. 実験

1 回目の実験では、ネットワークプログラミングに関する簡単な講義の後、各自簡単な実習を通してネットワークプログラミングの基礎に関して学ぶ。次に、各学生毎に自由にネットワークを利用したプログラミングの作成をおこなう。どのようなプログラムを作成するかに関しては、必要であれば、担当教員や TA と相談しながら進める。実際にどのような課題を作成するかを決定したら、プロポーザルを作成する。プロポーザルでは、どのようなプログラムを作成し(どのようなクラスを作成するか)、最終的にどのようなデモをおこなうかについて説明する。

また、本実験では、最後に各自が作成した課題に関するスライドを作成する。スライドは、各自が作成した課題をどのように決定したかの過程、プログラムの説明、プログラムのデモストレーション、プロポーザルで作成する予定だったプログラムと実際に作成したプログラムの違い、違いはなぜ生じたかに関する説明をおこなう。

本実験は以下のようなスケジュールでおこなう。

実験 1 回目の 1 週間後

ネットワークプログラミングに関する簡単な講義と演習

作成するプログラムの概要決定とプロポーザルの作成

演習プログラムの実行結果と取り組む課題のプロポーザルをレポートにまとめる。

レポートは pdf ファイルであること。

実験 2 回目まで

各グループの中間報告では、その時点で作成したクラスの説明、進捗状況、今後の予定をスライドにまとめる。スライドの ppt ファイルであること。

実験 3 回目締め切りまで

各グループが作成した課題のプログラムのアーカイブと課題を発表するスライドを提出すること。スライドは pdf ファイルであること。

実験 3 回目の 1 週間後まで

各グループの作成した課題と各自の取り組みに関してレポートにまとめること。また、課題他の学生が作成したスライドと自分のスライドを比較した考察をレポートとしてまとめる。レポートは pdf ファイルであること。

4. 報告

レポートには、名前、学籍番号を必ず記載すること。

5. 参考文献

●Java ネットワークプログラミング全般

[1] Elliotte Rusty Harold、戸松豊和監訳 “Java ネットワークプログラミング 第 2 版”

オライリー・ジャパン

- [2] クリフォード・J・バーグ “アドバンスド JAVA ネットワーキング” ピアソン・エデュケーション

●ソケットの基礎、Unix/C プログラミング

- [3] 羽山博監修、金内典充、今安正和共著 “UNIX ネットワークプログラミング” オーム社
- [4] W.リチャード・スティーブンス、篠田陽一訳 “UNIX ネットワークプログラミング 第2版 Vol.1” ピアソン・エデュケーション


```

import java.io.*;
import java.net.*;

public class JabberServer {
    public static final int PORT = 8080; // ポート番号を設定する.
    public static void main(String[] args)
        throws IOException {
        ServerSocket s = new ServerSocket(PORT); // ソケットを作成する
        System.out.println("Started: " + s);
        try {
            Socket socket = s.accept(); // コネクション設定要求を待つ
            try {
                System.out.println(
                    "Connection accepted: " + socket);
                BufferedReader in =
                    new BufferedReader(
                        new InputStreamReader(
                            socket.getInputStream())); // データ受信用バッファの設定
                PrintWriter out =
                    new PrintWriter(
                        new BufferedWriter(
                            new OutputStreamWriter(
                                socket.getOutputStream())), true); // 送信バッファ設定
                while (true) {
                    String str = in.readLine(); // データの受信
                    if(str.equals("END")) break;
                    System.out.println("Echoing : ");
                    out.println(str); // データの送信
                }
            } finally {
                System.out.println("closing...");
                socket.close();
            }
        } finally {
            s.close();
        }
    }
}

```

図 5 JabberServer.java

```

import java.io.*;
import java.net.*;

public class JabberClient {
    public static void main(String[] args)
        throws IOException {
        InetAddress addr =
            InetAddress.getByName("localhost"); // IP アドレスへの変換
        System.out.println("addr = " + addr);
        Socket socket =
            new Socket(addr, JabberServer.PORT); // ソケットの生成
        try {
            System.out.println("socket = " + socket);
            BufferedReader in =
                new BufferedReader(
                    new InputStreamReader(
                        socket.getInputStream())); // データ受信用バッファの設定
            PrintWriter out =
                new PrintWriter(
                    new BufferedWriter(
                        new OutputStreamWriter(
                            socket.getOutputStream())), true); // 送信バッファ設定
            for(int i = 0; i < 10; i++) {
                out.println("howdy " + i); // データ送信
                String str = in.readLine(); // データ受信
                System.out.println(str);
            }
            out.println("END");
        } finally {
            System.out.println("closing...");
            socket.close();
        }
    }
}

```

図 6 JabberClient.java